JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ProcessBuilder.Redirect

java.lang.Object
    java.lang.ProcessBuilder.Redirect

Enclosing class:

ProcessBuilder

```
public abstract static class ProcessBuilder.Redirect
extends Object
```

Represents a source of subprocess input or a destination of subprocess output. Each `Redirect` instance is one of the following:

- the special value `Redirect.PIPE`
- the special value `Redirect.INHERIT`
- the special value `Redirect.DISCARD`
- a redirection to read from a file, created by an invocation of `Redirect.from(File)`
- a redirection to write to a file, created by an invocation of `Redirect.to(File)`
- a redirection to append to a file, created by an invocation of `Redirect.appendTo(File)`

Each of the above categories has an associated unique `Type`.

Since:

1.7

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static enum`	**`ProcessBuilder.Redirect.`**	The type of a `ProcessBuilder.Redirect`.

Field Summary

Fields

Modifier and Type	Field	Description
`static final ProcessBuilder.Redir`	**`DISCARD`**	Indicates that subprocess output will be discarded.

static final ProcessBuilder.Redir	**INHERIT**	Indicates that subprocess I/O source or destination will be the same as those of the current process.
static final ProcessBuilder.Redir	**PIPE**	Indicates that subprocess I/O will be connected to the current Java process over a pipe.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
static ProcessBuilder.Redir	**appendTo**(File file)	Returns a redirect to append to the specified file.
boolean	**equals**(Object obj)	Compares the specified object with this Redirect for equality.
File	**file**()	Returns the File source or destination associated with this redirect, or null if there is no such file.
static ProcessBuilder.Redir	**from**(File file)	Returns a redirect to read from the specified file.
int	**hashCode**()	Returns a hash code value for this Redirect.
static ProcessBuilder.Redir	**to**(File file)	Returns a redirect to write to the specified file.
abstract ProcessBuilder.Redir	**type**()	Returns the type of this Redirect.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Field Details

PIPE

```
public static final ProcessBuilder.Redirect PIPE
```

Indicates that subprocess I/O will be connected to the current Java process over a pipe. This is the default handling of subprocess standard I/O.

It will always be true that

```
    Redirect.PIPE.file() == null &&
    Redirect.PIPE.type() == Redirect.Type.PIPE
```

INHERIT

```
public static final ProcessBuilder.Redirect INHERIT
```

Indicates that subprocess I/O source or destination will be the same as those of the current process. This is the normal behavior of most operating system command interpreters (shells).

It will always be true that

```
    Redirect.INHERIT.file() == null &&
    Redirect.INHERIT.type() == Redirect.Type.INHERIT
```

DISCARD

```
public static final ProcessBuilder.Redirect DISCARD
```

Indicates that subprocess output will be discarded. A typical implementation discards the output by writing to an operating system specific "null file".

It will always be true that

```
    Redirect.DISCARD.file() != null && // is the filename appropriate for the op
    Redirect.DISCARD.type() == Redirect.Type.WRITE;
```

Since:

Method Details

type

`public abstract ProcessBuilder.Redirect.Type type()`

Returns the type of this `Redirect`.

Returns:

the type of this `Redirect`

file

`public File file()`

Returns the `File` source or destination associated with this redirect, or `null` if there is no such file.

Returns:

the file associated with this redirect, or `null` if there is no such file

from

`public static ProcessBuilder.Redirect from(File file)`

Returns a redirect to read from the specified file.

It will always be true that

```
Redirect.from(file).file() == file &&
Redirect.from(file).type() == Redirect.Type.READ
```

Parameters:

`file` - The `File` for the `Redirect`.

Returns:

a redirect to read from the specified file

to

`public static ProcessBuilder.Redirect to(File file)`

Returns a redirect to write to the specified file. If the specified file exists when the subprocess is started, its previous contents will be discarded.

It will always be true that

```
    Redirect.to(file).file() == file &&
    Redirect.to(file).type() == Redirect.Type.WRITE
```

Parameters:

`file` - The `File` for the `Redirect`.

Returns:

a redirect to write to the specified file

appendTo

`public static ProcessBuilder.Redirect appendTo(File file)`

Returns a redirect to append to the specified file. Each write operation first advances the position to the end of the file and then writes the requested data. Whether the advancement of the position and the writing of the data are done in a single atomic operation is system-dependent and therefore unspecified.

It will always be true that

```
    Redirect.appendTo(file).file() == file &&
    Redirect.appendTo(file).type() == Redirect.Type.APPEND
```

Parameters:

`file` - The `File` for the `Redirect`.

Returns:

a redirect to append to the specified file

equals

`public boolean equals(Object obj)`

Compares the specified object with this `Redirect` for equality. Returns `true` if and only if the two objects are identical or both objects are `Redirect` instances of the same type associated with non-null equal `File` instances.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is the same as the obj argument; `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

Returns a hash code value for this `Redirect`.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this `Redirect`

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.